

Detailed Routing in OpenRoad

Armin Settels

22.09.2026

Where in the flow is Detailed Routing

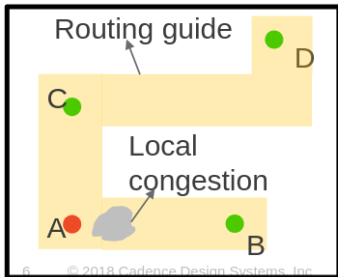


Global routing

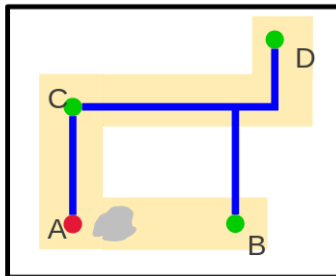
- capacity + congestion model
- outputs routing guides per net

Detailed routing

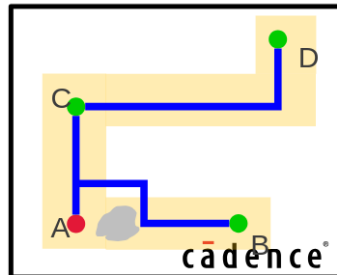
- exact geometry on routing grid
- considers DRC-rules



Guide-violation solution

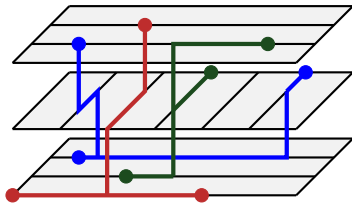


Guide-honoring solution

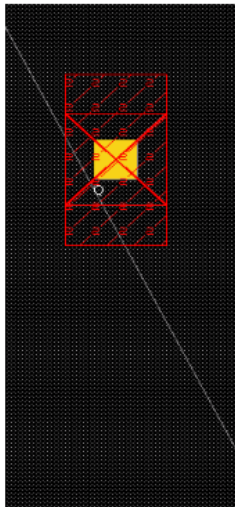


Detailed Routing

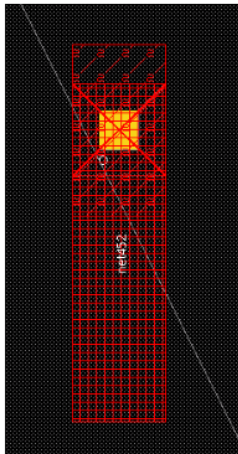
- After global routing
- **Task:** connect pins in each net by building a Steiner tree
- Using wires and vias in a 3-dimensional graph consisting of layers
 - **Wires:** Edges parallel to the layers
 - **Vias:** Edges connecting the layers
- Has to follow the design rules
- **Goal:** Minimize net length, errors, timing, ...



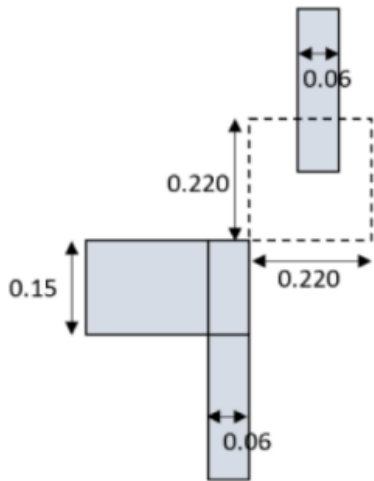
Min-area violation



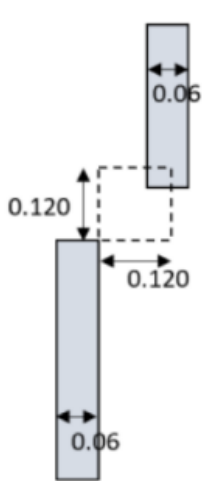
Min-area -violation-free



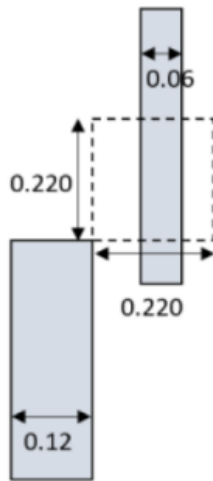
DRC Rules - Corner-To-Corner Spacing



X

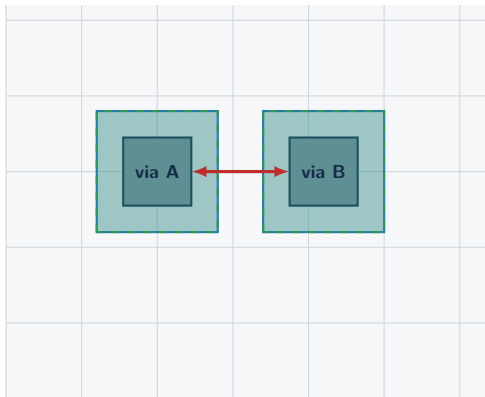


X



O

DRC Rules - Via-to-Via Spacing



Why this is algorithmically hard

- congestion
- complicated DRC rules
- millions of nets
- timing considerations
- weird pin access
- choices for wire width, via type

OpenROAD strategy

Decompose the problem, route locally, detect violations, then iteratively repair.

Formal view of detailed routing

Input

- LEF/DEF: layers, tracks, pins, blockages, placed instances
- global route guides: allowed/preferred regions per net and layer
- DRC rules
- possible wire / via widths

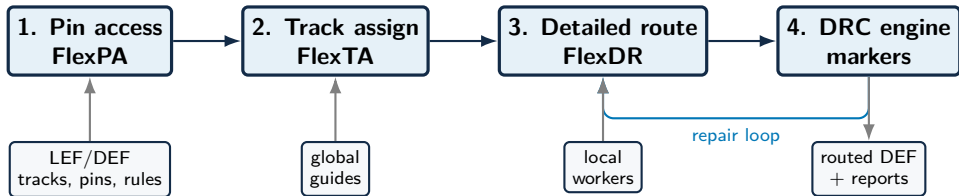
Optimization view

For each net n with pins P_n , choose geometry R_n such that

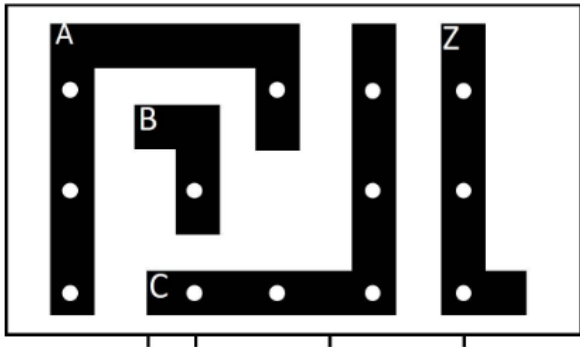
$$\text{CONNECTED}(R_n, P_n), \quad R_n \cap R_m = \emptyset \ (n \neq m), \quad \text{DRC}(R_1, \dots, R_k) = 0.$$

and timing clean and ...

OpenROAD's Detailed Routing



Step 1: Pin access



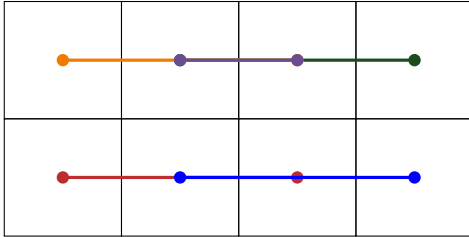
Problem

- a pin is a polygon, not a graph node
- legal contact depends on DRC-rules

FlexPA output

- enumerate access points per pin
- dynamic program to choose compatible ones

Step 2: Track assignment



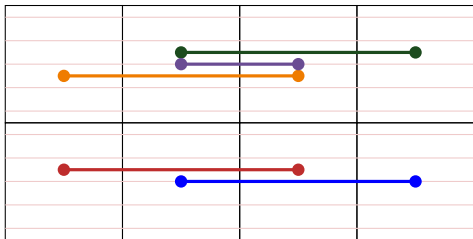
FlexTA does

- choose tracks inside guide regions
- reduces search space

Algorithm

- dynamic program
- minimize conflicts / penalties

Step 2: Track assignment (after)



FlexTA does

- choose tracks inside guide regions
- reduces search space

Algorithm

- dynamic program
- minimize conflicts / penalties

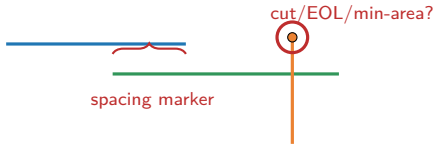
Step 3: Maze Routing

- A* shortest path search
- iteratively build Steiner tree
- parallelize over different nets

Path cost, conceptually

$$c(e) = \lambda_{wl} \ell(e) + \lambda_{via} [e \text{ is via}] + \lambda_{guide} [e \notin \text{guide}] + \lambda_{drc} \text{markerRisk}(e) + \dots$$

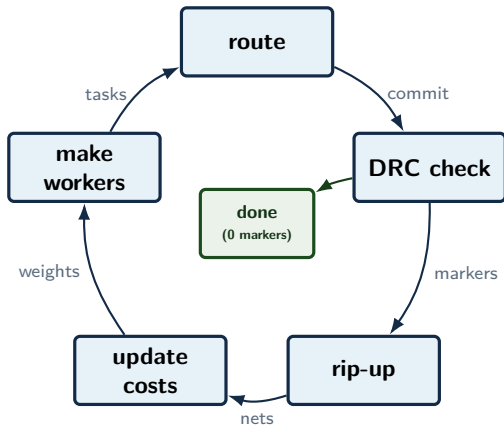
Step 4: DRC Markers



Markers encode

- rule type
- violating shape pair/region
- affected nets
- worker region to revisit

Search and repair loop



Each iteration

- Route nets in parallel workers
- Run full DRC check
- Collect violation markers
- Rip up offending nets
- Raise history costs near violations

Problem: Everything is heuristic - No guarantees!

Steiner Tree Search - Research in BonnRoute

Problem: non-optimal solutions due to path searches

Example: **global routing** and **detailed routing** of a net

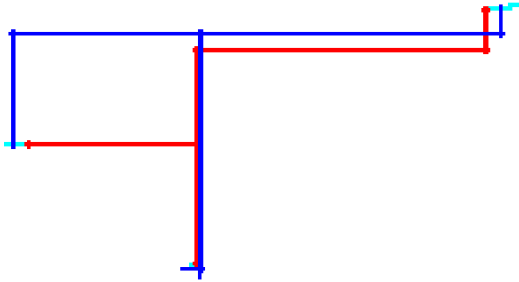
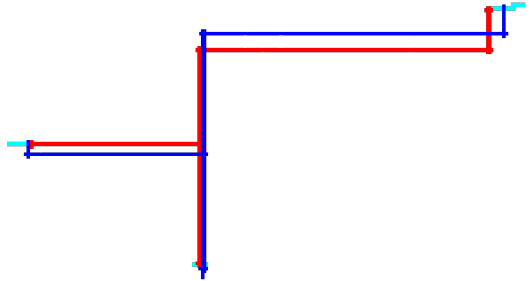


Figure: result of path searches

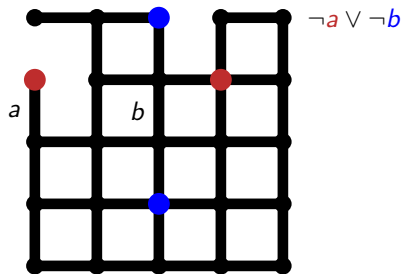


result of Steiner tree search

Idea: A* generalizes to Steiner trees

Decision-Guided SAT-Routing - Research in BonnRoute

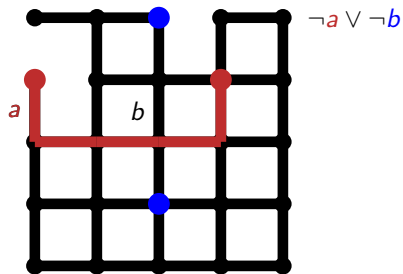
- Override internal SAT decision strategy
- Set variables based on shortest path searches
- Backjumps in SAT solver $\rightarrow$ rip-up nets
- SAT solver logic learns $\rightarrow$ **complete** solver



Order: red, blue

Decision-Guided SAT-Routing - Research in BonnRoute

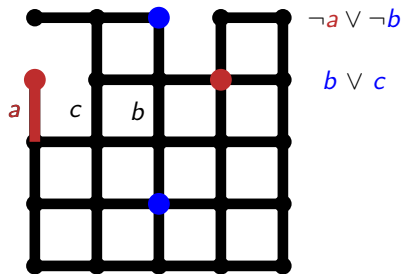
- Override internal SAT decision strategy
- Set variables based on shortest path searches
- Backjumps in SAT solver $\rightarrow$ rip-up nets
- SAT solver logic learns $\rightarrow$ **complete** solver



Order: red, blue

Decision-Guided SAT-Routing - Research in BonnRoute

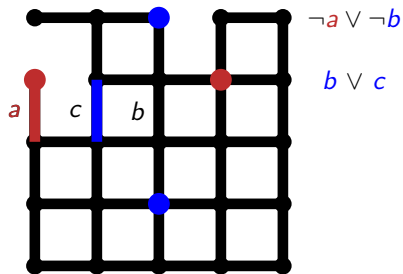
- Override internal SAT decision strategy
- Set variables based on shortest path searches
- Backjumps in SAT solver $\rightarrow$ rip-up nets
- SAT solver logic learns $\rightarrow$ **complete** solver



Order: blue, red

Decision-Guided SAT-Routing - Research in BonnRoute

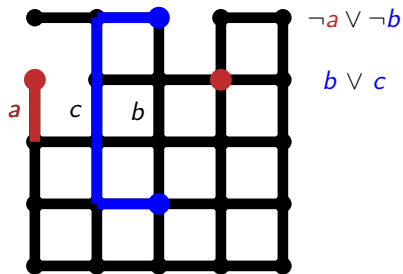
- Override internal SAT decision strategy
- Set variables based on shortest path searches
- Backjumps in SAT solver $\rightarrow$ rip-up nets
- SAT solver logic learns $\rightarrow$ **complete** solver



Order: blue, red

Decision-Guided SAT-Routing - Research in BonnRoute

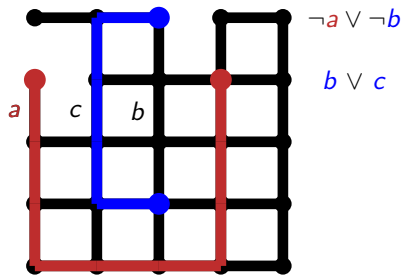
- Override internal SAT decision strategy
- Set variables based on shortest path searches
- Backjumps in SAT solver $\rightarrow$ rip-up nets
- SAT solver logic learns $\rightarrow$ **complete** solver



Order: blue, red

Decision-Guided SAT-Routing - Research in BonnRoute

- Override internal SAT decision strategy
- Set variables based on shortest path searches
- Backjumps in SAT solver $\rightarrow$ rip-up nets
- SAT solver logic learns $\rightarrow$ **complete** solver



Order: blue, red

Commands

OpenROAD Detailed Routing

```
read_guides route.guide  
detailed_route
```

Sources

- ISPD 2018 Benchmarks: <https://www.ispd.cc/contests/18/>
- ISPD 2019 Benchmarks: https://www.ispd.cc/contests/19/ISPD19_contest_result.pdf
- TritonRoute: <https://vlsicad.ucsd.edu/Publications/Journals/j133.pdf>
- The Tao of PAO: Anatomy of a Pin Access Oracle for Detailed Routing
<https://dl.acm.org/doi/pdf/10.5555/3437539.3437720>